

TechSaúde — Relatório Técnico de Desenvolvimento: Tela de Histórico de Consultas

FATEC Barueri — Gestão de TI | Escopo: front-end de historico_consultas.html (sem banco de dados/API)

1. Stack e arquitetura

Arquivo único e autocontido (historico_consultas.html), sem build step, sem dependências de terceiros além de fontes via CDN (Google Fonts: Poppins e Inter). Reaproveita o mesmo design system (variáveis :root, paleta creme/verde e componentes de cartão arredondado) já usado nas telas de tela-inicial e login, garantindo consistência visual entre as telas do app. Divisão interna:

- **HTML5 ~97 linhas** — marcação semântica com três regiões (.topbar, .sidebar, .content), sidebar de filtros e um container #list que é populado dinamicamente via JavaScript (sem cartões fixos no HTML).
- **CSS3 ~250 linhas** — variáveis de tema (--cream, --green-800, --teal, --gold, --red-muted), grid responsivo controlado por flex-direction (column → row a partir de 920px) e um segundo breakpoint em 560px para o grid interno dos cartões (.card-meta).
- **JavaScript ~220 linhas** — vanilla JS em escopo de módulo simples (sem IIFE explícita, sem dependências externas); mantém o array consultas como fonte única de dados (mock, em memória) e uma função central render() responsável por filtrar, ordenar e desenhar a lista a cada interação.

2. Histórico de iterações

- **v1** — Elaborado a partir das referências visuais de tela inicial e login já existentes (TELA_INICIAL e Tela_Login), extraindo o padrão de cor creme/verde, tipografia e formato de cartão arredondado, ainda sem tela de histórico definida nas referências originais — layout de lista foi uma extensão do design system, não uma réplica de mockup.
- **v2** — Decisão de escopo: em vez de uma lista estática, optou-se por um modelo de dados (consultas[]) com campos fixos (especialidade, profissional, unidade, localização, data, hora, status, observação), permitindo simular filtros e estados reais sem back-end.
- **v3** — Introdução do padrão .card / .card.open, com expansão de detalhes controlada por classList.toggle e sincronização de aria-expanded, para exibir o campo de observações sem navegação de página.
- **v4** — Adição da sidebar de estatísticas (updateStats()) e dos três filtros combináveis (busca textual, especialidade, situação), todos aplicados em tempo real dentro da mesma função render().
- **v5** — Ajuste de acessibilidade de teclado nos cartões (tabindex="0", role="button", listener de keydown para Enter/Espaço) e criação do estado vazio (#emptyState) para os casos em que a combinação de filtros não retorna resultados.

3. Responsividade

Sem depender de uma estrutura .site/.site-aside pré-existente (a tela não parte de um mockup de duas colunas fixo); a responsividade foi construída ponto a ponto para esta tela:

Breakpoint	Layout	Implementação
≥ 920px	Sidebar (320px) + conteúdo lado a lado	<pre>.layout{ flex-direction: row }, .sidebar{ width: 320px; flex-shrink:0 }</pre>

Breakpoint	Layout	Implementação
< 920px	Sidebar empilhada acima do conteúdo	<code>.layout{ flex-direction: column }</code> (padrão mobile-first)
≥ 560px	Metadados do cartão em grid 2 colunas	<code>.card-meta{ grid-template-columns: 1fr 1fr }</code>
< 560px	Metadados do cartão em coluna única	<code>.card-meta{ grid-template-columns: 1fr }</code> (padrão)

O container `.app` tem `max-width: 1180px` e é centralizado no viewport, com fundo em gradiente radial verde escuro ocupando toda a body, replicando o efeito de "cartão flutuante" já usado nas telas de login/boas-vindas.

4. Lógica client-side

Modelo de dados — cada consulta é um objeto simples (especialidade, profissional, unidade, localizacao, data, hora, status, obs); não há normalização em múltiplas tabelas, pois trata-se de um mock em memória sem persistência.

Filtros combináveis — a função `render()` aplica three `.filter()` encadeados sobre o array consultas (especialidade exata, status exato, texto livre em profissional/unidade/especialidade via `.includes()` case-insensitive), seguidos de `.sort()` decrescente por data+hora (`new Date(data+"T"+hora)`).

Preenchimento dinâmico de opções — o `<select>` de especialidade não é hardcoded: é populado no carregamento a partir dos valores únicos (`[...new Set(...)]`) presentes no array de consultas, evitando divergência entre filtro e dados.

Formatação de data — função utilitária `formatDate()` converte a data ISO (AAAA-MM-DD) em formato por extenso em português (DD de mês de AAAA · Dia-da-semana), sem uso de bibliotecas externas de data (ex.: `date-fns`, `moment`), apenas `Date` nativo e arrays de meses/dias mapeados manualmente.

Cartão expansível — o clique (ou Enter/Espaço) alterna a classe `.open`, que controla via CSS (`max-height + transition`) a exibição do bloco de observações, sem re-renderizar o DOM.

Estado de resultado zero — quando o array filtrado tem `length === 0`, o container `#emptyState` é exibido e a lista é limpa, evitando manter cartões órfãos na árvore DOM.

5. Acessibilidade (WCAG-aligned)

- Contraste de texto sobre fundo creme e sobre fundo verde-escuro dentro da faixa recomendada (AA) para os pares de cor definidos no design system.
- Cartões de consulta navegáveis por teclado (`tabindex="0"`, `role="button"`) com estado (`aria-expanded`) sincronizado à interação.
- Indicador de foco visível (`:focus-visible` com `outline` em `--teal`) nos cartões e nos campos de filtro.
- `<label>` associado a cada campo de filtro (`for/id`), inclusive nos `<select>`, para leitura correta por leitor de tela.
- Áreas de toque dos cartões e botões dimensionadas para uso confortável em dispositivos móveis, alinhado ao princípio de "letras grandes e fáceis de ler" citado nas telas anteriores do app.

6. Organização de assets

Reutiliza a mesma logomarca do padrão TechSaúde, porém recriada como **SVG inline** (gradiente turquesa/verde + traço estilizado de pulso), em vez de referenciar um arquivo de imagem externo — decisão que elimina uma dependência de asset binário e garante nitidez em qualquer resolução de tela. Nenhum outro ícone é carregado de fora: todos os ícones de metadado (unidade, localização, calendário, relógio, chevron) são SVGs inline no próprio JavaScript, dentro do objeto `icons`.

7. Testes executados

Verificação manual dos três filtros isolados e combinados entre si, do comportamento de ordenação por data/hora, da expansão/colapso de cartões via mouse e teclado, e da transição de layout nos breakpoints de 920px e 560px (redimensionamento de viewport). Não há suíte de testes automatizados (Playwright/Jest) implementada nesta rodada — a tela opera inteiramente com dados mock, sem chamadas de rede a testar.

8. Fora de escopo

Persistência real das consultas (banco de dados/API), paginação para grandes volumes de dados, ordenação por outros critérios além de data/hora, exportação do histórico (PDF/CSV) e sincronização com agenda do profissional/unidade não estão implementados nesta camada — a tela opera em modo protótipo com dados fictícios fixos no próprio arquivo JavaScript. Essas integrações serão cobertas em relatório próprio quando a camada de back-end for conectada, seguindo o mesmo tratamento já dado às telas de login e recuperação de senha.